KOENIG

step forward



JPA and Hibernate Mappings

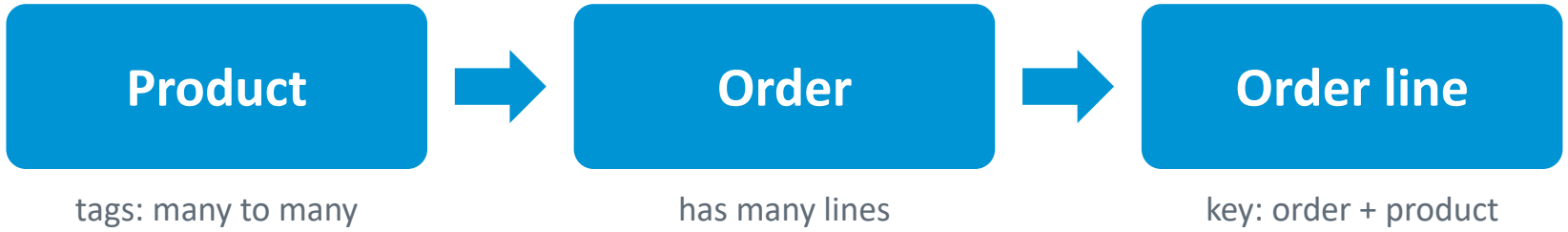
Spring Boot: relationships, keys, value types, indexes

What You Will Learn

1. Why objects and tables differ
2. Orders and lines: relationships, cascade, orphan removal
3. Tags and links: many-to-many, composite keys
4. Value types and enums
5. Loading: lazy, N+1 and join fetch
6. Indexes: fast searches

Demo Scenario: ShopKart Grows Its Model

ShopKart sells products. A customer places an order with lines and searches 50 000 products. We map this with Hibernate inside a small REST API and look at the tables and the SQL it creates.



What can go wrong

- **Lost lines**
- **Slow searches**
- **Too many queries**

Objects Do Not Fit Tables

```
class Order(
    val customer: String,
) {
    val lines: MutableList<OrderLine>
}
```

```
orders      (id, customer)
order_line  (order_id, product_id,
             quantity)

a row points with a key, not a list
```

Graph: objects reference each other and hold collections.
Table: flat rows, linked by keys, no list in a cell.
Mapping: the rules that translate one into the other. We write them as annotations.

An object points to other objects; a table row holds only values

JPA, Hibernate and Spring Data

```
@Entity
class Product(
    var sku: String,
    var name: String,
)
```

```
interface ProductRepository :
    JpaRepository<Product, Long>

repository.save(product)
repository.findById(1)
```

JPA: the standard annotations (@Entity, @Id, @OneToMany) in jakarta.persistence.
Hibernate: the engine that reads them and writes the SQL.
Spring Data: an interface becomes a repository; we write no SQL for save and find.

Three layers of help: a standard, an engine, a shortcut

The Layers of the ShopKart API

```
@RestController
class ProductController(
    private val service: ProductApplicationService
)
    fun create(@Valid request: ProductRequest)
```

```
@Service
class ProductApplicationService(
    private val products: ProductRepository
) {
    @Transactional
    fun create(r: ProductRequest): ProductResponse
```

Controller: thin. Takes the request DTO, calls one application service, returns a response DTO.
Application service: the use case and the transaction. It maps entities to DTOs while the session is open.
Domain: entities and repositories. An entity never leaves the service as JSON.

Controller -> application service -> domain -> MySQL

Seeing the Schema and the SQL

```
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.generate_statistics=true
```

```
Hibernate: create table product (
  id bigint not null auto_increment,
  sku varchar(255), ... )
Hibernate: insert into product
  (category,name,price,sku,stock)
```

create: drop and create the tables at each start, for demos and tests only.
show-sql: prints every statement. The header X-Statements of our API counts them per request.
Check: after each step we also look at the real table in MySQL.

We read what Hibernate writes, every time

Kotlin Setup: plugins and allOpen

```
plugins {
    kotlin("plugin.spring") version "2.4.10"
    kotlin("plugin.jpa") version "2.4.10"
}

allOpen {
    annotation("jakarta.persistence.Entity")
}
```

allOpen: Kotlin classes are final by default. Hibernate needs open entity classes to create lazy proxies.
plugin.jpa: adds the no-argument constructor that Hibernate needs to entities and embeddables.
plugin.spring: opens Spring beans, so @Transactional and other proxies work.

Small lines in build.gradle.kts that make JPA work with Kotlin

The Prepared Helper Classes

```
class BusinessException(
    val kind: ErrorKind, message: String
)

@RestControllerAdvice
class ApiExceptionHandler { ... }
```

```
@Component
class StatementCountFilter(...) {
    // adds header X-Statements
}

data class ErrorResponse(error, message)
```

BusinessException: the application service throws it with a kind (NOT_FOUND, CONFLICT, BAD_REQUEST). It never sees an HTTP status.
ApiExceptionHandler: the one place that turns exceptions into 404, 400 and 409 answers, also database and version conflicts.
StatementCountFilter: adds the header X-Statements to every response, the number of SQL statements of that request. We read it in the loading demo.
ErrorResponse: the small JSON body of every error.

Pasted at the start of the demo, used in every step

Demo: A Class Becomes a Table

What we do

- **Create:** an empty Kotlin Spring Boot project.
- **Connect:** MySQL in Docker.
- **Map:** Product, one REST endpoint.
- **Send:** POST and GET requests.

What it shows

- **Table:** written by Hibernate.
- **Statements:** create and insert.
- **Layers:** controller, service, DTO.
- **Errors:** 404 and 400 answers.

ManyToOne: The Foreign Key

```
@Entity
class OrderLine(...) {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    var order: Order? = null
}
```

```
order_line
  id
  order_id  --> orders.id
  product_name
  quantity
```

Many lines, one order: every line has one order, an order has many lines.
Foreign key: the column order_id lives in the table of the line.
LAZY: the order is loaded only when somebody asks for it.

The many side owns the column that points to the one side

OneToMany with mappedBy

```
@Entity
class Order(...) {
    @OneToMany(mappedBy = "order")
    val lines: MutableList<OrderLine> =
        mutableListOf()
}
```

```
mappedBy = "order"

means: the column is already described by OrderLine.order.
Order has no column for it.
```

mappedBy: tells Hibernate that the other side owns the foreign key.
Without it: Hibernate would create a second, useless join table.
First try: saving an order does not save its lines yet. We see that in the demo.

The list is a view; the foreign key stays on the other side

Helper Methods Keep Both Sides in Step

```
fun addLine(line: OrderLine) {
    lines.add(line)
    line.order = this
}

fun removeLine(line: OrderLine) {
    lines.remove(line)
    line.order = null
}
```

Why: only the owning side (line.order) writes the foreign key; the list is what you read.
Rule: always change a relationship through the helper, never one side alone.
Result: the objects in memory and the rows in the table tell the same story.

Set both ends in one place

Cascade: Pass Operations to the Children

```
@OneToMany(
    mappedBy = "order",
    cascade = [CascadeType.ALL]
)
val lines: MutableList<OrderLine>
```

```
orders.save(order)
  insert into orders ...
  insert into order_line ...
  insert into order_line ...
```

Cascade ALL: persist, merge and remove go from the parent to the children.
Use it for children that belong to one parent, like order lines.
Not for things that live on their own, like a product: deleting an order must not delete products.

Save the order, and its lines are saved too

Orphan Removal

```
@OneToMany(
    mappedBy = "order",
    cascade = [CascadeType.ALL],
    orphanRemoval = true
)
```

```
order.removeLine(line)

without orphanRemoval:
  update order_line set order_id=null
with orphanRemoval:
  delete from order_line
```

Without it: removing the line from the list only sets order_id to NULL and the row stays.
With it: a child without a parent is an orphan and Hibernate deletes it.
Cascade and orphan removal together make the parent the owner of the whole life of its children.

A line that leaves the order is deleted, not just unlinked

Demo: Orders and Lines

What we do

- **Map:** Order and OrderLine.
- **Lose:** lines without cascade.
- **Cascade:** lines saved with the order.
- **Remove:** a line, with and without orphanRemoval.

What it shows

- **Foreign key:** order_id in order_line.
- **Silent loss:** a response that lies.
- **Inserts:** one per line.
- **Delete:** only with orphanRemoval.

ManyToMany and the Join Table

```
@ManyToMany
@JoinTable(
    name = "product_tag",
    joinColumns = [JoinColumn(name = "product_id")],
    inverseJoinColumns = [JoinColumn(name = "tag_id")],
)
val tags: MutableSet<Tag> = mutableSetOf()
```

```
product_tag
  product_id  tag_id
  1           1
  1           2
  2           1

tag "wireless" exists once
```

Many to many: a product has many tags and a tag belongs to many products.
Join table: two foreign keys, nothing else. Neither side has a column for the other.
Owner: the side with @JoinTable writes the pairs.

A third table holds the pairs

Set or List?

Set: no duplicates; adding the same tag twice changes nothing. The right choice for many-to-many.
List without order: a bag. Hibernate cannot tell the rows apart, so it deletes the whole join table content and writes it again on a change.
equals and hashCode: a Set needs them. Compare entities by a stable business value, like the sku or the tag name, not by the generated id.
Kotlin: write equals and hashCode by hand on the sku, or use a data class for values that have no identity.

The collection type decides what Hibernate does

Demo: Tags

What we do

- **Map:** Tag and the join table.
- **Tag:** two products with one tag.
- **Repeat:** the same tag twice.
- **Look:** at the join table.

What it shows

- **Third table:** product_tag.
- **Shared row:** one tag, two products.
- **Set:** no duplicate pair.
- **equals:** by sku and name.

A Link With Data: the Order Line

```
class OrderLine(
    val order: Order,
    val product: Product,
    var quantity: Int,
    var unitPrice: BigDecimal,
)
```

```
order_line
  id  order_id  product_id  quantity
  1   1         1           1
  2   1         2           2
  3   1         1           1   <-- twice!
```

Association entity: when the link between two entities has data of its own (quantity, price), make the link an entity.
Own id: with a generated id nothing stops the same product twice in one order.
But: a line is really identified by two things together, the order and the product.

The line is the link between an order and a product, with a quantity

Composite Keys with @EmbeddedId

```
@Embeddable
data class OrderLineId(
    val orderId: Long = 0,
    val productId: Long = 0,
) : Serializable
```

```
@Entity
class OrderLine(
    @EmbeddedId
    val id: OrderLineId,
    ...
)

PRIMARY KEY (order_id, product_id)
```

Key class: an @Embeddable that holds the two parts. It has no table of its own; its columns join the primary key.
@EmbeddedId: uses it as the id of the entity. The pair must be unique, so the same product cannot be twice in one order.
Repository: JpaRepository<OrderLine, OrderLineId>; findById needs the whole key object.

Two columns, one identity: no extra id column

Key Rules: equals, @MapsId, Column Order

```
setOf(OrderLineId(1, 1))
    .contains(OrderLineId(1, 1))
// data class: true
// plain class: false
```

```
@MapsId("orderId")
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "order_id")
val order: Order
```

equals, hashCode, Serializable: two keys with the same values must be equal. A Kotlin data class gives it; in Java use a record.
@MapsId: the key column order_id is also the foreign key to the order. One column, two jobs.
Column order: Hibernate chooses it (by type, then name). Read show create table: the first column is the one that serves searches.

A key is an identity: compare by value, share the foreign key, read the order

Demo: Order Line Key

What we do

- **Link:** OrderLine points to Product.
- **Duplicate:** same product twice, own id.
- **Key:** OrderLineId as @EmbeddedId.
- **Refuse:** the API and the database.

What it shows

- **Own id:** two rows for one product.
- **Primary key:** (order_id, product_id).
- **409:** from the service.

Embeddable Value Types

```
@Embeddable
data class Money(
    val amount: BigDecimal,
    val currency: String = "INR",
)
```

```
@Embedded
@AttributeOverride(name = "amount",
    column = Column(name = "price_amount"))
@AttributeOverride(name = "currency",
    column = Column(name = "price_currency"))
var price: Money
```

No table, no id: the columns live in the owner's table.
Used twice: price on Product and unitPrice on OrderLine, with different column names through @AttributeOverride.
A value: two Moneys with the same amount and currency are the same.

A small object whose fields become columns of the owner

Enums: ORDINAL or STRING

```
enum class OrderStatus {
    PLACED, PAID, SHIPPED, CANCELLED
}

@Enumerated(EnumType.STRING)
var status: OrderStatus
```

```
default (ORDINAL):  status = 2
STRING:             status = SHIPPED

insert a constant in the middle:
  every stored number now means
  something else
```

ORDINAL (the default) stores the position. Reordering or inserting breaks old rows without any error.
STRING stores the name. Renaming is a data migration, reordering is free.
Rule: always write @Enumerated(EnumType.STRING).

Store the name, not the position

Demo: Money and Status

What we do

- **Embed:** Money as price and unit price.
- **Enum:** status as ORDINAL.
- **Fix:** status as STRING.
- **Compare:** the column in MySQL.

What it shows

- **Columns:** of the owner, no money table.
- **Overrides:** same type, two sets of names.
- **Stored:** 2 first, then SHIPPED.

LAZY and EAGER

```
@ManyToOne(fetch = FetchType.LAZY)
var order: Order? = null

@OneToMany(mappedBy = "order")
val lines: MutableList<OrderLine>
```

```
LAZY:  loaded on first use
        (a proxy stands in)
EAGER: loaded together with
        the owner, always
```

Defaults: @OneToMany is LAZY and ManyToOne is EAGER. Write LAZY on every ManyToOne.
LAZY is right: load only what the use case needs.
Cost: touching a lazy relation later costs one statement, and it needs an open session.

When is the related data loaded?

N+1 and LazyInitializationException

```
orders.findAll().map { it.toResponse() }

// 1 statement for the orders
// +1 per order for its lines
// +1 per new product
```

```
without a transaction:
  LazyInitializationException
  Cannot lazily initialize
  collection ... (no session)
```

N+1: a list of 100 orders costs 201 statements, because each order loads its lines when the mapping touches them.
LazyInitializationException: a lazy relation touched after the session is closed.
Fix the second by mapping to DTOs inside the application service transaction.

The two surprises of lazy loading

join fetch: Load What the Use Case Needs

```
@Query("select distinct o from Order o " +
       "join fetch o.lines l " +
       "join fetch l.product")
fun findAllWithLines(): List<Order>
```

```
before: 7 statements
after:  1 statement

other endpoints stay LAZY
```

join fetch: loads the relation in the same SQL statement, with a join.
distinct: removes the duplicate orders that the join creates.
Per use case: fetch what the endpoint needs, nothing more. An @EntityGraph does the same without writing the query.

One statement, the data we will use

Demo: Loading Orders

What we do

- **List:** orders with lines.
- **Count:** X-Statements = 7.
- **Break:** no transaction.
- **Fix:** join fetch, 1 statement.

What it shows

- **N+1:** one per order and product.
- **Exception:** lazy outside the session.
- **Fetch:** per use case.

Declaring Indexes

```
@Table(
    name = "product",
    indexes = [
        Index(name = "uk_product_sku",
              columnList = "sku", unique = true),
        Index(name = "idx_product_category_price",
              columnList = "category, price_amount"),
    ],
)
```

```
EXPLAIN ... where sku = ...
  type=const  rows=1

EXPLAIN ... category + price
  type=range  rows=100

no index: type=ALL rows=50000
```

Unique index: the fastest lookup and a rule enforced by the database (a duplicate sku gives error 1062, our API answers 409).
Composite index: one index on several columns, in the order you list them.

Single, composite and unique, in @Table

Demo: Indexing

What we do

- **Search:** category and price.
- **Break:** price without category.
- **Measure:** index sizes.

What it shows

- **Unique:** duplicate sku gives 409.
- **Cost:** indexes bigger than data.

Key Takeaways

1. **Relationships**
   The many side owns the foreign key; cascade and orphan removal care for the children.

2. **Links**
   Set for many-to-many; a link with data is an entity with a composite key.

3. **Values**
   Embeddable Money and STRING enums.

4. **Loading**
   LAZY by default, map to DTOs in the service, join fetch per use case.

5. **Indexes**
   Index what you search and check EXPLAIN;



Happy Learning :)